DS	**Session**	Graph	**Question Information**	Level 1 Challenge 90

Question description

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city.

What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Constraints

- $2 \le n \le 500$
- $1 \le m \le 1000$
- $1 \le a,b \le n$

Input

The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered 1,2,…,n. The bank is located at crossing 1, and the harbor is located at crossing n.

After this, there are m lines that describe the streets. Each line has two integers and b: there is a street between crossings a and b. All streets are two-way streets, and there is at most one street between two crossings.

Output

First print an integer k: the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
1 3
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
2
1 4
3 4
```

Test Case 2

INPUT (STDIN)

```
4 5
1 1
2 2
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
1
1 4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int h)
```

Test Case 2

KEYWORD

```
int bfs(int n,int s,int t)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
735
```

Test Case 3

NLOC

```
93
```

Code

police_chase.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 505
#define MAXM 2005

typedef struct {
    int u, v;
} OriginalEdge;

typedef struct {
    int to, cap, flow, rev;
} FlowEdge;

FlowEdge fedges[4 * MAXM];
int fhead[MAXN], next_fe[4 * MAXM], fe_cnt = 0;
OriginalEdge orig[MAXM];

void link(int i, int h) {

    (void)i; (void)h;
}

void add_flow_edge(int from, int to, int cap) {
    fedges[fe_cnt] = (FlowEdge){to, cap, 0, fe_cnt + 1};
    next_fe[fe_cnt] = fhead[from];
    fhead[from] = fe_cnt++;

    fedges[fe_cnt] = (FlowEdge){from, cap, 0, fe_cnt - 1};
    next_fe[fe_cnt] = fhead[to];
    fhead[to] = fe_cnt++;
}

int parent_edge[MAXN];

int bfs(int n, int s, int t) {
    for (int i = 1; i <= n; i++) parent_edge[i] = -1;
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = s;
    parent_edge[s] = -2;

    while (qh < qt) {
        int u = q[qh++];
        if (u == t) return 1;
        for (int e = fhead[u]; e != -1; e = next_fe[e]) {
            int v = fedges[e].to;
            if (parent_edge[v] == -1 && fedges[e].cap - fedges[e].flow > 0) {
                parent_edge[v] = e;
                q[qt++] = v;
            }
        }
    }
    return 0;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    for (int i = 1; i <= n; i++) fhead[i] = -1;

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        orig[i] = (OriginalEdge){u, v};
        add_flow_edge(u, v, 1);
    }

    int max_flow = 0;
    while (bfs(n, 1, n)) {
        for (int v = n; v != 1; ) {
            int e = parent_edge[v];
            fedges[e].flow += 1;
            fedges[fedges[e].rev].flow -= 1;
            v = fedges[fedges[e].rev].to;
        }
        max_flow++;
    }

    bool visited[MAXN] = {false};
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = 1;
    visited[1] = true;
    while (qh < qt) {
        int u = q[qh++];
        for (int e = fhead[u]; e != -1; e = next_fe[e]) {
            int v = fedges[e].to;
            if (!visited[v] && fedges[e].cap - fedges[e].flow > 0) {
                visited[v] = true;
                q[qt++] = v;
            }
        }
    }

    printf("%d\n", max_flow);
    for (int i = 0; i < m; i++) {
        int u = orig[i].u;
        int v = orig[i].v;
        if ((visited[u] && !visited[v]) || (visited[v] && !visited[u])) {
            printf("%d %d\n", u, v);
        }
    }
    link(1, 1);
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc police_chase.c -o police_chase
PS C:\C Programming> .\police_chase
4 5
1 2
1 3
2 3
2 4
3 4
2
2 4
3 4
PS C:\C Programming>